



SMART-CONTRACT SECURITY AUDIT

Submitted contract

pasted-source

100 / 100

A · ROBUST · LOW RISK

ENGINES RUN

3 / 6

CONSENSUS FINDINGS

1

CRITICAL / HIGH

0 / 0

RAW FINDINGS

1

LINES OF CODE

503

SCAN TIME

20.0s

Executive summary

Audit Forge analyzed 503 lines of Solidity with 3 independent engines (mythril, semgrep, solhint). After cross-engine consensus clustering, 1 distinct issues were identified — 0 of which were independently confirmed by two or more engines.

SEVERITY	TOTAL	MULTI-ENGINE	WEIGHT
■ INFO	1	0	single-engine

Confidence signal: a finding flagged by several independent engines is far more likely to be real than one raised by a single tool. Single-engine findings — especially style/lint rules and findings inside audited library dependencies — frequently include false positives and should be triaged before acting.

Engine errors• slither: Slither produced no JSON output

- aderyn: Aderyn produced no JSON report: WARNING: Error loading config file: /root/.docker/config.json: open /root/.docker/config.json: permission denied

Failed to Derive AST & EVM Info: Encountered invalid solc version in src/Contract.sol:

Findings (1)

INFO

1. Use Nested If

/input/Contract.sol:312 · semgrep (1 engine) · confidence low

Using nested is cheaper than using && multiple check combinations. There are more advantages, such as easier to read code and better coverage reports.

Engines

Audit Forge orchestrates these independent open-source tools and reconciles their output by consensus. Full credit to their authors; each runs under its own license. Details and links: auditforge.org/#/engines

Slither Static analysis · v0.10.4 · by Trail of Bits · not run

Mythril Symbolic execution · v0.24.8 · by Consensys Diligence

Aderyn AST analysis · v0.5.5 · by Cyfrin · not run

Semgrep Pattern matching · v1.85 · by Semgrep, Inc.

Solhint Linting · v5.0.5 · by Protofire

Echidna Property fuzzing · v2.2.4 · by Trail of Bits · not run

Methodology

Each engine ran in an isolated, network-disabled, resource-capped Docker sandbox. The contract was analyzed statically and symbolically in isolation — no deployment-time or runtime context was simulated. Findings from each engine were normalized to a common schema (SWC IDs as the cross-reference taxonomy) and clustered by category, file, and line proximity (± 3 lines). A finding's "engine count" indicates how many independent engines agreed; the 0–100 score is a severity-weighted roll-up that gives more weight to multi-engine consensus.

DISCLAIMER · This report is produced by automated analysis and AI synthesis. It automates the first pass and is not a substitute for a professional manual audit; a high score is not a guarantee of safety. Automated engines produce false positives — particularly single-engine, style/lint, and library-dependency findings. Audit Forge makes no warranty as to completeness, accuracy, or fitness for purpose. Deploying to a public blockchain is irreversible; engage a qualified auditor for any contract handling user funds.

POWERED BY CONDO

Support the cause

Audit Forge is free and open source, built and operated by Condo, the onchain treasury protocol. Every scan costs real compute. If this report saved you an audit fee, support the cause with a contribution so the engines stay running and free for the next builder.

CONTRIBUTION ADDRESS · ANY EVM CHAIN

0xEf3E49a3197417ccDbF5F6A60D89f7Fa4823199d

Send ETH or tokens on Ethereum, Base, Arbitrum, Robinhood Chain or any EVM network. Contributions are voluntary and non-refundable.

Audit Forge is a Condo product. Learn more at condobase.io/home/auditforge